Overview

- Kernel modules must be rewritten sparingly to maintain a stable migration surface area, enhancing long-term compatibility.
- Token instantiation should generate new objects, avoiding excessive reliance on interfaces, for improved efficiency.
- Limit token functionalities to 3-4 core operations such as mint/burn and transfer, ensuring simplicity in execution.
- A Rust-based approach is favored for the runtime execution environment, utilizing a single file compiled to WebAssembly for efficiency.
- Key-value storage model ensures complete separation between state and storage, enhancing data integrity and access efficiency.
- Access Control Lists (ACL) managed at the operator level are essential for scalability and maintaining transaction integrity.
- ZK proofs will replace traditional consensus mechanisms, advancing scalability by proving transaction inclusion without full consensus.
- Current Ethereum scaling bottleneck requires extensive RISC-V cycles, segmented across clusters for proof generation.
- Inclusion proofs provide a dynamic method to validate token states against the latest database conditions, improving verification speed.
- Technical constraints such as Blake hash overhead necessitate innovative solutions to optimize performance and execution cycles.

Notes

🧩 Token Architecture & Design Philosophy

- Kernel modules should be written once with tightly controlled changes to maintain a fixed surface area for migration.
- Token instantiation should create new objects rather than implementing against interfaces and deploying new classes.
- Avoid creating new tables for every token in relational DB perspective - instead use encompassing token contracts.

- Token functions limited to 3-4 core operations: mint/burn, tokenize/detokenize, transfer, and transfer with checks.
- Transfer with checks passes token metadata to determine required hooks for invocation.

⚙️ Runtime Execution Environment

- Runtime execution takes invocation context with pre-state of accounts and scheduler-acquired locks on individual keys.
- Access intent requires pre-declaring keys for updates, with scheduler acquiring necessary locks.
- Rust-based approach preferred: Single Rust file with libraries compiled to WebAssembly for proof generation.
- Stateless contract design eliminates EVM storage access complications and proof generation headaches.
- Code and data separation - load specific data into memory rather than embedding in runtime.

💾 Storage Model & State Management

- Key-value storage model adopted with complete separation between state and storage.
- Runtime loads specific keys into memory for execution rather than accessing storage directly.
- Trust assumption on operator for database reads and ACL calls against specific users/tokens.
- Transaction receipts include pre-state and post-state of every key involved in transactions.

🔒 Access Control & Permission Management

- ACL checks handled at operator level in trusted manner for scalability.
- Runtime execution environment limited to signature checks for authenticity verification.
- Complex permission flows and matching algorithms must happen at operator level, not in execution environment.

- User and token configs stored as actual data in storage, not embedded in runtime.

🔍 ZK Proofs & Verification Requirements

- ZK proofs essential for proving correct state transitions from DB state A to state B without consensus.
- ZK proofs replace consensus mechanism by proving transaction inclusion and correct execution.
- Current scaling bottleneck: Ethereum blocks require ~2^30 RISC-V cycles, broken into 2^20 cycle shards.
- Parallel GPU processing used with 200-400 cluster cores for proof generation.

📜 Inclusion Proofs & Accumulator Schemes

- Two design paradigms: inclusion proofs vs Bitcoin-style model with historical block references.
- Inclusion proofs allow proving current token state against current DB state.
- Lattice-based accumulators promising but unproven for ZK-VM compatibility due to Blake hash overhead.
- Verkle trees identified as potentially viable option using Pedersen commitments.
- RSA accumulators deemed too cumbersome for ZK-VM implementation.

⚡ Technical Constraints & Performance

- Blake hash overhead: 1 Keccak hash requires ~2^18 RISC-V cycles.
- Memory bottlenecks limit execution to 2^20 cycle chunks in current implementations.
- Streaming Jolt implementation in development to handle full blocks without sharding.

Action Items

Abhirup

- ☐ Confirm lattice-based accumulator viability with algebraic hashes by consulting with Justin

- [ ] Draft technical proposal incorporating discussed ideas and share with team before email distribution
- [ ] Design minimal Rust execution environment with token contract functionality using Merkle/Verkle trees
- [ ] Include Karan in proposal discussions and coordinate joint email to team

Transcript

S
Speaker 1
00:00
Yeah.
S
Speaker 2
00:00
Can you repeat what you just said?
S
Speaker 1
00:03
Yeah. See, I think one important consideration is that the way we are thinking about these kernel modules is that you write them once and you tightly control it so that any new changes, or if tomorrow, if we decide to change something.
S
Speaker 2
00:19
Yes.
S
Speaker 1
00:20
A surface area for migration is very fixed, well contained. Right. So from that perspective, if you were to create a new token. I would much rather have a new token to be an instantiation of a new object versus implementing against an interface

and deploying a new class itself. So the equivalent here is that if you are to think about it from your relational DB perspective, you are creating a new table for every token that you paid, which doesn't make sense, correct?

S

Speaker 2

00:57

Yeah. So I was thinking of more of it as like just borrowing like the contract structure and then editing it for some whatever use case that we want. Like we want like a token contract that is sort of, well, encompassing and then sort of deploy it and then essentially keep using it for maintenance transfer.

S

Speaker 1

01:17

But you, Yeah, I mean the. You can. My suggestion would be to. Don't make the assumption that you will write this in solidity for each of this. If you were to think of these as like independent trades or trust trades, your token probably has three functions, maybe four functions. Right. One is a min burn or tokenized detokenized transfer and transfer with checks. Right. And so when you do transfer with checks, you are also actually passing the token and based on the token from the metadata, you could figure out what the, you know, what other hooks you are expected to invoke.

S

Speaker 2

02:04

Yeah, so if I'm doing so, I would also have a flow where I'm. So there are two cases. So I just purely want to check. So, so inside the token I want to purely check. For example, say I want to check that there exists some other kind of a token. This can be done, right?

S

Speaker 1

02:22

You mean a read, right?

S

Speaker 2

02:23
A read of a token for some against the user? Basically.
S
Speaker 1
02:28
Yeah, yeah, it's fine. Why would you want. I don't think you would even want to go via the program for that because you could just directly query the database, right?
S
Speaker 2
02:40
Yeah. Then how would I simulate it in the. How would I simulate this in the runtime execution environment? So for me to simulate it in the runtime execution. So this is my point. So if I have to simulate a runtime check, then that couldn't. That cannot be a check that the operator performs independent of the execution environment. Because.
S
Speaker 1
03:01
So I think this is part of the runtime consideration. Right. So in, for example, now we are talking about defining the exec function.
S
Speaker 2
03:09
Yes.
S
Speaker 1
03:10
The runtime is a glorified exec and the exec just doesn't just take the transaction. Right. It actually takes an invocation text.
S
Speaker 2
03:17
Yes.
S

Speaker 1
03:19
So the invocation context should have the pre state of the account and from the scheduler's perspective, it's the schedulers when it is scheduled, it would have also acquired the required locks on these individual keys. If you're planning to update the balance of an account, you. You would have acquired a lock.
S
Speaker 2
03:42
Okay, okay, fine. Sure, sure. Understood. So essentially you would take some kind. So for example, I want to say that this user should be able to mint this token. So then you would basically take the value field of that user also to check against whether the user essentially has these things and then the sort of.
S
Speaker 1
03:59
Checked again in that design. Right. When I talk about access intent, that's what I mean. When you pre declare the keys that you want to.
S
Speaker 2
04:12
I understood. Yeah, so I understood. And then basically transfer with checks is essentially just transferring inside the single the same token. But now say if I have transfer with checks and in the checks I want to access other kinds of tokens, then that should also be possible, right?
S
Speaker 1
04:27
Yeah. The expectation is that you would have send. You do not want to compute that on the fly because you are. I mean if you had to compute that in the runtime, you are adding over it computation overhead, you would much rather pass that in externally.
S
Speaker 2
04:44

Okay, fine. Okay, good. So this. So essentially. I think so my point. So you. Your suggestion would be that I would just take a Rust and I'll just define like a trade for my token contract with these three four kinds of functions. And then I can just basically compile my Rust program. I'll have my execution environment purely in Rust and I'll just basically compile it down to Rust 5 and generate my proofs.
S
Speaker 1
05:08
Yeah, exactly. So you'll have a very access just one single Rust file with all the libraries added. So you generate object.
S
Speaker 2
05:24
Yes.
S
Speaker 1
05:24
Right. Yes. And that's what you're proving against. And also it's very important that this entire contract itself.
S
Speaker 2
05:31
Yes.
S
Speaker 1
05:32
Is stateless, right?
S
Speaker 2
05:34
Yes.
S
Speaker 1
05:35

Now Ethereum, because evm, because of all the technical debt it has, it also has to account for, you know, your. When you're. You have to work around basically the fact that in your. Your storage access, how do you do it? How do you generate proofs over there?

S

Speaker 2

05:57

Yes.

S

Speaker 1

05:57

It becomes like a huge headache. Except here it's like as long you know what state it was and someone your loader basically has signed and given it to you. Right.

S

Speaker 2

06:07

Yeah. So like I understand. So you are.

S

Speaker 1

06:11

So.

S

Speaker 2

06:11

You are of. So actually this makes my life much easier because going by the EVM route is actually much harder for me because I now have to sit and basically design. So this is actually like very easy because actually we have done this also like on our end. Not like.

S

Speaker 1

06:25

Yeah, I mean, I come from a similar mindset. Not even mindset. Also from a design perspective, this makes a ton of sense. Right. Like you always. Your code and data is separate and whatever data that you want to load.

S

Speaker 2

06:42

Yes.

S

Speaker 1

06:42

You load it. You do not add that into the runtime. That makes no sense.

S

Speaker 2

06:48

Yes, correct. So this is exactly what I was also thinking. So. So initially this is what I thought and then basically APUR was reply made me think that probably I might have to go to the evm because everybody. But I. I think we now agree.

S

Speaker 1

07:00

No, I. I don't think that's even a requirement. Like, I mean, why are we inheriting Internet is not an E. L2.

S

Speaker 2

07:07

Yes. It's not. This is what I believe.

S

Speaker 1

07:10

Or even any. Any specific L2 for that matter. Then it makes no sense for us to.

S

Speaker 2

07:15

Yeah. Or like. Or an L1 for that matter. Like, it's just.

S

Speaker 1

07:19

Yeah.

S

Speaker 2

07:20

It's its own.

S

Speaker 1

07:21

Basically the. The fundamental argument is that like the. The blockchain ecosystem has evolved in such a. In a specific direction.

S

Speaker 2

07:28

Yes.

S

Speaker 1

07:29

And with the specific set of design trade offs.

S

Speaker 2

07:33

Yes.

S

Speaker 1

07:33

None of those apply to us.

S

Speaker 2

07:35

Yes.

S

Speaker 1

07:36

So it makes no sense for us to inherit those design trade offs.

S

Speaker 2

07:43

Yes.

S

Speaker 1

07:43

We are operating in a place where we are kind of like forced to use whatever exists.

S

Speaker 2

07:50

Yes. Perfect. So essentially now. So now this is clear. So I can just create a Rust con. I can basically create a Rust environment with preliminary functionalities being just this token main transaction transfer and essentially just denied. Then. Then I have a DB state. Right. And this is good enough. This is what my understanding is.

S

Speaker 1

08:09

Yeah. Yeah. I mean this is my understanding as well. Because this drastically simplifies even the kernels that we'll have to write.

S

Speaker 2

08:16

Yes. So you don't think there is a need for account contracts?

S

Speaker 1

08:22

I think it will be required at some point because the ACL whatever checks that you want to do, the only way these checks make sense at scale when you have a lot of competing transactions which want to happen one after the other.

S

Speaker 2

08:40

Yes.

S

Speaker 1

08:40

Is if you have to do your post checks directly in the runtime itself. In order to do that, you will need a ACL module. So my point is ACL module at the kernel. So where all this token specific configs and user specific configs you also store as actual data on the act. The finte storage itself.

S

Speaker 2

09:12

Yeah, so this is what I'm like not able to see like. Like we discussed in the like just about some time ago. I could just basically pass checks as part of my runtime exec call.

S

Speaker 1

09:24

And then that works for pre. Like pre checks. Right. Like for example after a state update. So the checks that you want to do.

S

Speaker 2

09:33

Yes.

S

Speaker 1

09:34

Are. There is a mistake in the. In our terminology. For example, when we see pre. You know pre commit and post. You know pre hooks and post hooks.

S

Speaker 2

09:46

Yes.

S

Speaker 1

09:47

Actually those are not. You only need one type of hook which is during the state of. Which is before the state update. Right. Pre you can simulate against the existing state and you can do it yourself. We don't want to add that to the runtime.
S
Speaker 2
10:01
Yes.
S
Speaker 1
10:01
Post doesn't make sense. Unless if you want to do it as a post, you would have actually sent it as a transaction bundle, right?
S
Speaker 2
10:10
Yes.
S
Speaker 1
10:11
So what you really want to do is for example before you flip a switch, you want to make very specific checks that pass this user pass kys. Whatever checks that you want to do you. You do it on. I mean you could also argue that you know what, even this could better done as a pre commit checks as well.
S
Speaker 2
10:41
Yeah, I mean this is my understanding. So I am not able to think of a case where I would want.
S
Speaker 1
10:47
Wait, there is a new email from. Let's see what he has said.
S
Speaker 2

10:56
Two quick points. I would prefer not to couple all parts. Unless we are purely aiming for a demo. Then no worries about coping. Otherwise we want four backend building blocks user accounts plus ECLs. So to go in third party APIs.
S
Speaker 1
11:09
Yeah, I think the proven should be simpler.
S
Speaker 2
11:12
Basically we take who what when our experience.
S
Speaker 1
11:15
Yes, he is exactly saying what we are discussing good.
S
Speaker 2
11:18
And simply have a mechanism to create.
S
Speaker 1
11:22
Yeah, this is exactly what we are discussing right now. Exactly.
S
Speaker 2
11:26
Using all inputs and outputs. So that remedy basically token managers only.
S
Speaker 1
11:31
Yes.
S
Speaker 2

11:32
Yes.
S
Speaker 1
11:33
Okay. Yeah. Yes.
S
Speaker 2
11:37
Okay, fine. Good. How do you do?
S
Speaker 1
11:39
Oh, there is a high five here. Oh that's.
S
Speaker 2
11:41
Yes, okay good. Yeah high five.
S
Speaker 1
11:44
Yeah.
S
Speaker 2
11:45
So I think this is like a good, good understanding of.
S
Speaker 1
11:50
I. I think now it is good.
S
Speaker 2
11:54

So essentially. So this is my understanding. So now. Okay, so now. So okay. So my. Yeah, so this is essentially what I want. So who, what when, how etc is out of the. Out of the kernel module for now at least.

S

Speaker 1

12:10

Like. Yeah, I am. I'm okay with it.

S

Speaker 2

12:15

Yeah. So my point is like signature checks. Yes. Make it part of the check, but the who, where and how. This is what my understanding was because when I heard Pramod say I was asking very specifically the LOS organizer, will they run the operator, they will have an acl. So. And I keep. Kept asking him this question because I wanted the requirement to be very clear because I did not. I thought that these permission flows would be very hard. That was the reason actually I was asking. That was my understanding actually of this.

S

Speaker 1

12:41

Yeah. So I think one thing though is that if you're operating some of these checks right.

S

Speaker 2

12:50

Yes.

S

Speaker 1

12:52

It's. You will reach a point depending on what the. You know, how many transactions we do per second and what people are act. You. You do not want to. You basically want to ensure that there is no read latency or, you know, you're reading a inconsistent state of the database.

S

Speaker 2
13:13
Yes.
S
Speaker 1
13:14
When you're doing the checks.
S
Speaker 2
13:15
Yes. Yeah. So inconsistent state of the database is not possible because you trust. So this is the. Having this trust assumption on the operator is very important. And to lay it down properly, the trust assumption is that the operator can read the database and make trusted ACL calls, basically. So it can basically make the call that do you have access to this token or can you read this access against a specific user for this token or this value of a token? So and so forth. Essentially what I'm trying to say. So this. You make an assumption that the node can actually do it.
S
Speaker 1
13:56
Got. Got it. Yeah, I mean that's. I'm okay with it for now.
S
Speaker 2
14:03
Yeah, I think for now. I think that's like the best strategy. And later on if you want to sort of include some kind of permissionless flows, then we'll have to think about it. But I think the only permissionless flows that I can think would because you're making a token state DB the ones that I can actually think of is just an existence of token basically because all that you have is like a token.
S
Speaker 1
14:27

The other one is, for example, when you want to enable on chain composability right where you want to execute, then this becomes very important. The moment you have like order books on Internet itself. I don't know if that's ever going to happen.
S
Speaker 2
14:41
Audible.
S
Speaker 1
14:44
Can you hear me now?
S
Speaker 2
14:45
Yeah, I can hear you now.
S
Speaker 1
14:48
Okay. Was there not audible for some time?
S
Speaker 2
14:50
Just for like a. It was not. Not audible. It was like you were slurring a bit. I don't know, like those. Yeah.
S
Speaker 1
14:58
Okay. Okay. So basically what I was What I'm trying to say is that if you also have, if you want to ensure, for example, especially when you're doing order books and you want to ensure the fastest matching.
S
Speaker 2
15:14
Yes.

S

Speaker 1

15:14

Then it would turn out that the best way to do it is also to execute this ACL checks on the runtime itself whenever that's not a consideration for now. For the use cases that we have discussed, this is enough.

S

Speaker 2

15:32

Yeah. So like I, I assume for scale to happen, all of this has happened. Need to happen at the operator level in a trusted way and then just the verification sort of happens. So for example, the order matching happens where like you, so suppose the order matching has happened and the check is happening that oh, this can be assigned to this. This is sort of programmatic. But running like a complex matching algorithm with like access logic is sort of not possible. So essentially what I want to say is like at the execution environment for scale to hit you have to pass correct inputs.

S

Speaker 1

16:10

Yeah, I agree with that.

S

Speaker 2

16:12

Yeah, so that's essentially my point.

S

Speaker 1

16:14

And you keep any, any verification of the inputs or even any of those is added overhead.

S

Speaker 2

16:24

Exactly right, exactly. So the only checks that you can keep at the execution environment are like signature checks essentially just like an authenticity of your against your ID basically in some sense.

S

Speaker 1

16:38

Yeah, yeah, that makes sense. Yeah, yeah. One consideration door is done then you do not. Maybe it's not important for now, but when user moves, for example, where does the user config live? Where do the token config live? For example, in some of these ACLs. Yes, it might be embedded direct. The expectation is that if you read the original paper, the expectation is that some of these checks are baked directly into the token. But I'm not sure how practical that is.

S

Speaker 2

17:13

Yeah, so this is my point. So the checks that are baked directly into the token, those checks have to be against existence of tokens. So for example, if I need like say EMI token or if I want a loan token, I could say that this has, this user is kyc. That means it has a KYC token. Let's assume for now this is. So you could sort of say that require that this token exists which can be done in the required existence.

S

Speaker 1

17:38

Yeah, I mean if you're able to do existence, you could also do like a conditional on the value, right?

S

Speaker 2

17:43

Yeah, you could do a. So I, I, I think of like.

S

Speaker 1

17:47

These then that, I mean if you build for that capability then if you're, if you build for the capability for existence, you Also build for the capability for conditional. Then you

have the ability to do ACLs natively or at the runtime. But assuming the ACL is against not, you know, checking a checking a specific field, not some complex logic.
S
Speaker 2
18:10
Yeah. So for example, I. So for example, whatever. So whatever. For example, whatever can if you want.
S
Speaker 1
18:18
To encode a flow, like if you have a startup equity.
S
Speaker 2
18:22
Yes.
S
Speaker 1
18:23
Before you transfer it from one to one person to another, do you actually want to get. Get it verified by the startup? Either it happens as a pre flight check where you are expected to send in their signature so that it goes through. Yes, that is reasonable. But then the fact when they. But then the fact that this expectation is there, that is acl.
S
Speaker 2
18:52
Yeah, My point is like, so how.
S
Speaker 1
18:55
Do you code that into the token program?
S
Speaker 2
18:58

Yeah, so this is what I was saying. So if I'm saying transfer against checks or just a pure check where I'm just passing the key value pair of the specific user and I'm just saying that now I checked against the value field that such tokens exist, then this can be programmatically embedded into the token program itself. But what cannot be embedded, which I don't see right now is essentially against a different user. So I am making a call and I am now I can pass maybe an additional and the action has to be taken for say. So I am making the call and I am like an admin and I am issuing a token.

S

Speaker 1

19:33

Unless the transfer with checks also takes as parameter the user config key.

S

Speaker 2

19:41

Yes, exactly. And then inside when I'm passing parameter I'm not just passing key, I'm also passing the value across the key and.

S

Speaker 1

19:51

Yeah, so you would actually load these keys.

S

Speaker 2

19:54

Yes, value in the exec environment and I check it against my DB that this was the state.

S

Speaker 1

20:01

Yeah, so that for example, if you also look at the transaction receipt design that I propose, it actually has the pre state and the post state of every account that every key that is involved. Yes, right. So the only in order to be able to produce that you will have to the runtime has to load that into the. Into the memory. And even before that the scheduler would have had to acquire the required logs, right?

S
Speaker 2
20:24
Yes. So the only thing that I'm like I want to be able because the value field will. Value field will be large. I will only sort of add specific field values that are necessary. So specific tokens that are necessary for the execution environment currently and then I should be able to prove that they existed in the value field of the user in the pre state.
S
Speaker 1
20:46
Yeah, yeah. So this would be like you're literally just loading. That would be limits on the amount of, you know, the size of the data in the value.
S
Speaker 2
20:55
Yes, yes, exactly. So the. So my point is I don't think of ACLS as these checks. I'm thinking of ACLs more as like an. So these are execution level checks, conditional checks. Like saying that does check.
S
Speaker 1
21:08
I mean, when I speak, when I think about ACL at the runtime, this is what I think. Think about. Right. Anything about that. Any complex logic that you want to do. Exactly where you don't have the ability to change this on the runtime, you still want to, you know, because of whatever changes at the token manager end, they should still be able to do that. So.
S
Speaker 2
21:30
Exactly, exactly.
S
Speaker 1
21:31

So that only happens via like a trusted API.

S

Speaker 2

21:35

Yes, exactly. So this is exactly what I. So this is my, essentially my thinking as well. So that. So, okay, so I can write. We can, I can come up with an independent trust execution environment for this. Fine. This is clear. Now let's fix the commitment thing that I wanted to fix.

S

Speaker 1

21:53

Yeah, yeah. I think this. We are, both of us are aligned, right?

S

Speaker 2

21:57

Yeah.

S

Speaker 1

21:58

So now we have like a clear storage model which we agree is a key value and then a clear runtime where you know, the state and the storage are completely separate. And then in order to execute, you actually load specific, whatever key that you want into memory. So now the biggest question for me is like, why do inclusion proofs matter?

S

Speaker 2

22:24

Why should inclusion proofs matter? Great question. So essentially, so there are two designs, okay? If you do inclusion proofs and if you don't do inclusion proofs, if you do inclusion proofs, then you can actually prove that your token exists. The current state of your token exists against the current state of the db.

S

Speaker 1

22:44

But I have signed transaction receipts, so.

S

Speaker 2

22:47

That'S my whole point. So if you don't do inclusion proofs, then it becomes like a bitcoin model. If you, in a bitcoin model, what you do is that you don't care about your deep. Your current state of the token being in the current state of the year token that you have right now. Being in the current state of the db, all that you care for is that there is a block that you know it could have existed maybe a year ago against which you can sort of say that your token was last executed.

S

Speaker 1

23:19

You get.

S

Speaker 2

23:20

So this, these are two different design paradigms and you have to choose which one is. Okay. It is a.

S

Speaker 1

23:25

For permissioned ones. I see where the transaction received may not work because. But then should that be allowed though? For example, if someone does a charge, one complication is if someone does a chargeback, but then even the chargeback should have a receipt. Right?

S

Speaker 2

23:46

Yeah. So if someone does a charge back, it has a receipt and essentially your token gets manipulated and you would essentially have that city, that transaction sitting on the block. Right. So you'll have to reflect it as a transaction. But actually the bigger problem is not this. The bigger problem is that if your DB keeps growing up. So the reason Ethereum actually went for inclusion proofs is that I don't want all nodes to

store the entire. Yeah. So if I might. If my last token was executed say two years ago and I want to prove. And by the two years, 500 GB of data has been generated, I need the last block from somewhere to prove my existence.

S

Speaker 1

24:30

But the question is do in this model.

S

Speaker 2

24:33

Yes.

S

Speaker 1

24:34

Are user expected to prove it? Yeah, because that is, I mean, very important consideration. Right. So the, because that kind of tells you. In my mind that's like a major scaling limitation. Because now the only way to do this is either you use Mercury trees, which have inclusion proofs.

S

Speaker 2

25:04

No, there's.

S

Speaker 1

25:05

So there you start using like variants of Merkle trees and then it like.

S

Speaker 2

25:11

I'll tell you three. So there is. So I've read now everything, I even read your accumulator thing. So even for both of them, I'll tell you the problem cases. So before let's postpone this decision and let me tell you about even the lattice based accumulator thing. So even if you don't go for the inclusion proof, let's say you don't go for the inclusion proof. So there are two problems that happen. One is if you go

for, let's say you just want like a pure accumulator scheme where you want to do batch updates and batch deletions. This is all that you care about. You don't care about membership. Basically.

S

Speaker 1

25:47

Yeah, because the reason why I'm leaning towards that is because you can actually parallelize this. And also in fact you can start doing like SIMD type of a thing where you have single instruction, multiple data, vectorize the whole thing and run very quickly.

S

Speaker 2

26:05

Yeah. So the problem with this is essentially that you. I don't so far, especially for the lattice ones, I don't know if you can snarkify it. Basically you can basically create a ZK proof for it. So essentially today the overheads of say, let's say take your lattice and Blake. Okay. So lattice and Blake is essentially you will have to do Blake hashes which becomes problematic. So if I'm saying including data points, I have a thousand data points, I have to do Blake hashes and then I have to do this G power thing on the. Like the lattice. So this is not. I'm not sure whether you can actually put it inside a ZKVM and it'll be efficient. So this is one.

S

Speaker 1

26:50

Yeah.

S

Speaker 2

26:51

Okay.

S

Speaker 1

26:51

Yeah, that's a. I. I would see you. Those are like two independent things. Like I'm still not convinced why we need. I. I agree. We need proofs. Not sure why those have to be unique approved.

S

Speaker 2

27:03

So we'll come back why don't need. So why we need ZK proofs. Let me tell you. So you. So when the runtime execution is being done, it is creating a new state of the db, right? New state of the db.

S

Speaker 1

27:16

Yeah.

S

Speaker 2

27:17

You. You don't know without ZK proof that the correct transactions have been used to actually create the new state of the DB and the transact there is. How do I say it?

S

Speaker 1

27:29

Like have the receipt. No, like for example, these. The. At any point if. If I have the receipts.

S

Speaker 2

27:36

Yes.

S

Speaker 1

27:37

And if I also have the ability to run that use my receipt and simulate it against the existing state, I. I should get the response. So it's like I can easily verify it by applying the changes.

S

Speaker 2
27:51
What is a receipt?
S
Speaker 1
27:53
A receipt has. For example, in a. In a typical database transaction, how do they do rollbacks? How do commit and rollbacks work? Before a transaction is committed, it actually generates this data structure where you have a pre state that is the state of the rows that you are accessing. It has that state after the code update has happened. After the state update has happened, it produces a new state. It also has a snapshot of that.
S
Speaker 2
28:30
Yes.
S
Speaker 1
28:31
Right. So if you want to roll back, you just take this and.
S
Speaker 2
28:35
Yeah.
S
Speaker 1
28:38
So my question is if the. If these are like well known programs, there are only six of them, handful of them and the state machine is well known, the updates are well known. I could just execute against it to.
S
Speaker 2
28:54

So just think about it just. So just think about the following thing. So I'm. I know. So you're right. I've been also thinking about this, but.

S

Speaker 1

29:01

There is a slight computer. Right. Like it's a very specific.

S

Speaker 2

29:08

Three minutes and I'll try to explain where the problem is. Okay. The problem is against the following. Suppose.

S

Speaker 1

29:15

I'm.

S

Speaker 2

29:15

Suppose you give me a transaction. So suppose checkers Check as executed, giving a transaction to the operator. Okay, now there is no consensus. Forget consensus for now, okay? Typically in a blockchain, a consensus is where you say that this transaction has actually been included, the ledger, and you say that the ledger is.

S

Speaker 1

29:32

Right.

S

Speaker 2

29:34

Okay? Now in the finternet world for now, we have not even entered into the consensus domain right now.

S

Speaker 1

29:40
Yeah, I mean that doesn't exist as long as.
S
Speaker 2
29:43
Here, wait a minute, let me complete that. So now think about the following case that I am saying that this transaction has been executed, okay? I am creating a new state of the DB without actually including the transaction. Where will you claim its existence? Say after five days that it actually exists in this state of db, which I have said. So you have to place a trust assumption in saying that I am modifying this so there is an additional trust assumption in. What you are saying is that my modification of the DB when I am saying that it is includes this thousand transactions. It actually includes this thousand transactions.
S
Speaker 1
30:25
But, but then, see, I have a receipt, but the receipt is.
S
Speaker 2
30:30
No, no, the receipt. What is the receipt? What is the. If I am going, if I am as an operator are just signing you and telling you that, look, I've included a transaction, then that is no receipt because you're placing a trust on me. What is the receipt? The receipt has to somehow come from the execution. So typically in a blockchain world, this is done by consensus by other nodes, attesting to the fact that now your transaction actually exists in the block. Because the moment you take consensus out of the picture, you need a ZK proof to say that, oh, I use these transactions to actually go from this state of the DB to this state of the db. That proof is essentially the receipt for all the transactions used from going this state to that state.
S
Speaker 2
31:16
And that essentially means that you were actually included in the block.
S

Speaker 1
31:23
Yes, but then wouldn't you be. I see what you mean. So basically, without ZK proofs, the way to do it would be that every address, every key is well known, the hash of the key is well known, and some hash of their every point in time, whatever their state was and their deltas, those are well known.
S
Speaker 2
31:44
So I mean, basically you will have to go and say that.
S
Speaker 1
31:47
Oh, like you'll have to do it for every single account that is.
S
Speaker 2
31:53
No, I'm saying that if there are thousand transactions that are included in the block, basically all the parties will have to check against it because why will they trust me otherwise? You will have to place a trust on the Operator saying that the operator is doing this. Correct. But then that loses the point, basically. So, essentially, the reason you need a ZK proof is for this reason that you want to claim that the DB has gone from state A to state B, including XYZ transactions. And this has this XYZ which transactions were used that actually caused the DB to go from state A to state B. This is the reason you use ZK snark. If you don't want to use ZK snark, the same statement is proven via consensus. Why consensus?
S
Speaker 2
32:36
You say that now I have a DB state A and I've gone to DB state B for this. I've used this sequence of transactions. Look, now this is what I'm claiming and the consensus, basically all my other nodes in the peer network would basically confirm that this. Okay, what Vineet is saying is correct. And we will all come to a common understanding of DB state B now. But in the absence of db, in absence of

consensus, in absence of repeating this computation, what I really want. What I want is I want to generate a ZK proof, put it on the blockchain, and let all the nodes on the blockchain conform to the fact that I've gone from DB state A to DB state B using these transactions. That's why you actually want a ZK proof for this.

S

Speaker 1

33:17

Help me understand this. See, let's say there are N number of keys in a database.

S

Speaker 2

33:24

Yes.

S

Speaker 1

33:24

Right?

S

Speaker 2

33:25

Yes.

S

Speaker 1

33:25

And either from transaction one to two, either the N goes from N to N, either the number N grows, or. That is secondary. Right? It's just that at any point in time you compute a hash, if you have all the inputs for the hash at any given point in time, anyone should be able to read, Sorry, I verify this. Right?

S

Speaker 2

33:50

I do not understand. Can you repeat?

S

Speaker 1

33:53

Basically what I'm saying is that if you have. For every time you produce a block, that is. Let's call it every fixed.

S

Speaker 2

34:03

Yes.

S

Speaker 1

34:03

Number of whatever. Millisecond. Right? 100 millisecond, 200 millisecond.

S

Speaker 2

34:08

Whatever concrete numbers I put, I generate a block of 50 transactions. Okay.

S

Speaker 1

34:14

You then. And it has 50 accounts. Right? The genesis block.

S

Speaker 2

34:20

Yes. So you have, let's say the DB at say, thousand accounts. Let's say the DB at thousand accounts. Let's say the db at thousand accounts. And I'm generating 50 transactions on this thousand accounts. All are transfers from account A, some. Some account X to some account Y. All are transfers.

S

Speaker 1

34:38

Yeah. So. And that must be at TN. That was DMA's. Have some hash. Right. If you have to all the thousand accounts.

S

Speaker 2

34:52

Yes.

S

Speaker 1

34:54

You hash their current value.

S

Speaker 2

34:56

Yes, I hashed all their current value together.

S

Speaker 1

34:59

And you combined them into one single hash value. Yeah.

S

Speaker 2

35:03

Okay.

S

Speaker 1

35:05

And then when after a transaction has happened again.

S

Speaker 2

35:09

Yes.

S

Speaker 1

35:10

So all the individual inputs are also hashed and published. Yes. Right.

S

Speaker 2

35:15

Yes. So now if you want to check.

S

Speaker 1

35:18
Against this as long as. Oh, okay. Because there is no. You don't know what did. What you hash to. There is no commit and reveal. There is only a hash commit.
S
Speaker 2
35:29
Yeah. So I mean, if you want to check against this, you will have to. You have to check. I'll have to give you all the data and you'll have to hash it again and you will have to check it. So classified verification become expensive.
S
Speaker 1
35:39
Yeah, understood. Makes sense. The reason why we even despite this, the reason why you use it is because you can actually commit.
S
Speaker 2
35:48
And yes, the reason I give them, I make a Merkle tree or like I make a Merkel tree is because now I can give a short proof against the leaf node.
S
Speaker 1
35:58
Yeah. I mean, Mercury argument after it has gone through consensus, you still have the block. It does. Right.
S
Speaker 2
36:04
The Merkle tree. So essentially Merkle tree is to. To give a shot. But the point that you generate a small proof for this is essentially to claim that against DB state A to DB state B, these transactions have been executed. They have been executed according to the computational environment that we all agree on. And this is so there is only computational.
S
Speaker 1

36:26
I agree with the. The commitment. Like you are committing to the. The inputs. Yes, right. But why does the. Why do you, why do you need verifiable compute? Because these are well known.
S
Speaker 2
36:38
So the verifiable compute, the state machine.
S
Speaker 1
36:41
Is well known, right?
S
Speaker 2
36:42
Yeah. So the state. But the state machine is well known. But I, If I only give you. So I am saying that, oh, the hundredth Fibonacci number is a million. Let's say, I don't know what it is, but let's say it's a million. Now I'm giving you. This is, I'm giving you inputs 100 and a million. There is no way for you to, you know, the algorithm of Fibonacci. The only way you can say that this is one the output is correct is to actually re execute it. Right? To re execute the program. I don't want all nodes to re execute the program.
S
Speaker 1
37:13
I want to generate nodes. But if you have the transaction listed for my state, if it is my state isolated, the storage for that is also isolated. For example, let's Say my token balance.
S
Speaker 2
37:23
You have to show that the output state was actually generated in the correct way. According to. I could create the output state out of thin air, right? I could claim something.

S

Speaker 1

37:33

Okay, the problem, okay, now I understand the problem here is that the operator could use a new version of the kernel module without publishing a specific exactly.

S

Speaker 2

37:46

New version, meaning a malicious model. I could just say that instead of transferring 50 to say why I'm transferring to my account itself.

S

Speaker 1

37:55

Okay.

S

Speaker 2

37:55

Okay. So I need to say that this is a pre agreed execution environment which I have followed and I have taken these sequence of transactions which has taken me from DB state A to DB state B. So what are my input? My input is the set of transactions DB state A, DB state B.

S

Speaker 1

38:15

Understood? Understood.

S

Speaker 2

38:16

And my snark proof or my ZKVM will basically show now that these set of inputs are consistent. That means I taking these set of transactions will lead me from DB state A to DB state B B if I follow my execution.

S

Speaker 1

38:29

Understood? Understood.

S

Speaker 2

38:31

So now the problem with the deep. So there is an inherent problem when you choose how you maintain the DP states. If you go via an accumulator scheme that is lattice. I don't know if ZKVMS is scaled. ZKVMS will not scale. That is my understanding right now. It might change in a week's time. But my understanding right now is that ZK VMs won't scale because it uses Blake hashes. I also checked whether it can use Poseidon hashes. There is no paper basically, but if you could use Poseidon hashes instead of Blake hashes, then there is no limiting factor. But I have to ask my other cryptography collaborators that if my understanding is right. But to. For my under. From my understanding is that there is no. Right now there is no putting lattice hashes into ZK VMs.

S

Speaker 1

39:16

Yeah, so one thing that I want to. When they say lattice, it's actually not lattice cryptography.

S

Speaker 2

39:26

I saw. No, it is. It is. It is. You essentially. So I read the paper, I saw the paper that actually proposed this.

S

Speaker 1

39:33

It takes like a 1024 Vector and then it's based the assumption that is that somehow that exists. You know, it's like if you do a wrapping add or a wrapping subtract it, somehow the U16 operates.

S

Speaker 2

39:56

So it is because of lattice properties. So I see this is proposed by researchers in Facebook. So it actually builds on a 1994 Primitive. I actually went and read the paper.

S

Speaker 1

40:06

So if U16 behaves like a lattice. Yes, then it is. It is lattice.

S

Speaker 2

40:11

Exactly. So the point is that. But you cannot. So there are some things. So I want to make sure. My understanding is that this should be. If this is possible, this should be possible with also Poseidon hashes over a prime field. But I'm not sure whether that is secure because the security for that has not been argued. So I'm not sure. And the crypto native community sort of looks down upon algebraic hashes for some reason. I don't know like Justin doesn't like it but. But I'm. I'm not sure. I'll. I'll ask around. Let me see if this is a viable option. But I'm not. But till then I think like my. So the. The.

S

Speaker 1

40:49

So this is the problem calculator doesn't solve for inclusion. Right. You can only know if it works.

S

Speaker 2

40:56

This is even modular. The accumulation scheme. I'm talking. If I just. If I do accumulation, I forget about inclusion proofs for now. And I'm okay with receiving a block that existed one year ago from somehow I maintain archiver nodes. Say all of this is possible.

S

Speaker 1

41:12

Yeah. I mean by regulations they would be required to maintain this. Right? Like maybe.

S

Speaker 2

41:19

Yeah. But let's say all of this is possible. But even then lattice based hashes like this lt@ basically I don't know how to. Like I don't think it'll scale under zkvm. So that is one problem. The second thing is there are.

S

Speaker 1

41:32

There's the other accumulators because of like specific things around Blake.

S

Speaker 2

41:37

Yeah. So Blake is essentially not F. So right now Blake itself will take a lot of. So if I'm not wrong, like 1 so for example, 1 KEK hash itself takes a lot of so inside an RVM like RISC V it takes about 2172 power 18 cycles.

S

Speaker 1

41:55

But I think that's also probably because these RISCPY VMs don't have. Okay, fine. That's not. You would still have to create a ZK circuit for it.

S

Speaker 2

42:05

Yes. So there are dedicated ZK circuits also for it like in SP1, but I don't know whether SP1 has dedicated Blake hashes. And if you end up doing so many Blake hashes then I don't know how much overhead it like it takes. Like my understanding is that the overhead is higher if I do normal binary hashes inside a vm. Okay.

S

Speaker 1

42:26

And got it.

S

Speaker 2

42:27
Yeah. So that is. That is exactly like. That is the reason I am sort of skeptical. But I can try it out. I can actually put an apply because I actually found this work Very interesting. I want to pursue this also, but it will take me some time to figure out whether I can do algebraic hashes or I can put it inside zkvm. It's not a question that I can like answer very quickly. Yeah. So that is one thing.
S
Speaker 1
42:48
Yeah.
S
Speaker 2
42:48
The second, the second accumulator scheme is basically the RSA accumulator scheme that cannot be put inside a zkvm. I have seen many people try it, but it's. It will be very cumbersome. The other one that I found, before.
S
Speaker 1
43:03
We head down that path, I think we need to get consensus on what inclusion proof is. I don't think we have discussed it.
S
Speaker 2
43:09
Okay, so inclusion proof, essentially. Whatever.
S
Speaker 1
43:12
No, I mean not when, I mean not us. But like in the last initially I.
S
Speaker 2
43:17
Was actually, I. I am very keen, I'm very keen on like actually proposing like a UTXO kind of model like you were saying. But the only thing that I'm sort of skeptical about

is like can I hold blocks of that far? Like maybe a year, two year. How so? If, if it's like I'm only going to verify my data in the past three months and everything needs to be renewed after six months or something, then it's fine.

S

Speaker 1

43:49

Because then like then it's a very easy idea to socialize. People know, for example, when you transact with a bank, you get a salon.

S

Speaker 2

44:00

Yes.

S

Speaker 1

44:01

You go to do you know, you go make a mutation in the property register.

S

Speaker 2

44:08

Yes.

S

Speaker 1

44:09

You get a Chalan.

S

Speaker 2

44:10

Yes.

S

Speaker 1

44:10

These are. You get like so many paper trials of everything that has happened.

S

Speaker 2

44:17
Yes.
S
Speaker 1
44:20
Telling people to, you know, trust a very spec for them. It's like a opaque ZK circuit. Right.
S
Speaker 2
44:32
I completely agree to your point. But the reason I was saying that it becomes immaterial because is that even as an accumulator. So the reason I was not focusing on inclusion proof is because I have come my conclusion right now is that the only way that I can do inclusion like do accumulator schemes are also using the best ways Verkal trees that can be put inside zk.
S
Speaker 1
44:54
Like how fast are worker trees?
S
Speaker 2
44:57
So you'll have to see it. I don't know how much gains because.
S
Speaker 1
45:00
Like when these guys considered replacement Solana, by the way, historically has never had a local tree. It was that they knew that was the one.
S
Speaker 2
45:16
I have been arguing against merkle trees for a long time because I, I have worked with merkle trees all my life. I know it doesn't scale inside a ZK set, but the Problem is with the worker.

S
Speaker 1
45:27
The.
S
Speaker 2
45:28
The approach that the accumulator hashes have like anything is they cannot be put inside a zk. So the only thing that I see is Verkal trees. Verkal trees are like the. It might. It might be slightly like less cumbersome inside a ZKVM because I have to just generate Pedersen commitments. Like I just need to basically just a group commitment along the path which may not be so hard. Like. So for example, If I have 256. So it's essentially saying that I have to generate Pederson's of 256. 256 times. So which is like saying that I want to do a Peterson commit of 2 power 8. A Peterson commit of 2 power 8 takes hardly any time like milliseconds. So I am doing two power eight times such things. It'll be like under. Under something like for.
S
Speaker 2
46:22
And I can parallelize this as well because I have to generate the eventual states. So that is essentially the whole point. But the point the thing is getting it inside a zkvm. I don't know how big the circuits would be but I'm for Pederson, for Pedersen. I. I think it shouldn't be so bad because an MSM of 2 power 8 will not be so bad also. And then I. I'll have to just see how. How big it becomes for the update. So right now I'm either. I'm essentially. This is the reason I'm basically.
S
Speaker 1
46:52
All of these EKVMs deal with. Deal with the site. Do they kind of don't do it at the ZK but instead like pass it as like external sign statements or hints into assigned inputs into the ZK. Vm.
S
Speaker 2
47:08

Sorry, deal with what?
S
Speaker 1
47:11
I mean, sounds like this is a scaling bottleneck for anyone putting ZK into production, right?
S
Speaker 2
47:18
Yes, it is. It's a very huge scaling bottleneck.
S
Speaker 1
47:20
So do they just deal with it or I wonder if it is possible for them to send this itself as like the skill input or state updates.
S
Speaker 2
47:32
Is it?
S
Speaker 1
47:34
Yeah, yeah.
S
Speaker 2
47:35
So the state updates is essential.
S
Speaker 1
47:36
So it produces the. Basically you are not. Or then you have consensus over it like you have. You generate state updates and proofs and then you verify these via re execution or it doesn't matter in the case of an L2 because like anyway you.
S

Speaker 2
47:54
So if they are centralized sequencers, ideally they are supposed to do state updates also. And it matters that the state updates are actually going to the zk. So it's not just that the execution environment is Checking the validity of the transaction, they're checking it against the state of the db. So ideally they are supposed to do it. So typically what do they do it?
S
Speaker 1
48:14
Do they do it though?
S
Speaker 2
48:15
Yes. So for example, an Ethereum block today from doing end to end, taking it from state A to state B and everything takes about two power 30, that's a billion riscycles.
S
Speaker 1
48:27
Okay.
S
Speaker 2
48:28
Okay. And the way they execute these proofs is now they break it into shards of length 2 power 20.
S
Speaker 1
48:35
Okay.
S
Speaker 2
48:36
They can't go beyond 2 power 20 execution lens because the memory bottlenecks it. So that is essentially one of the works that I did with Justin where we proposed like a

streaming jolt. Like we can do a streaming and that's what we implement with a 16Z research also. So that implementation is going to come out soon. It's just work in progress. So essentially you could now implement like an Ethereum block with our work without actually breaking it into starts. You could just basically take the whole block and just execute like generate proof on the fly.

S

Speaker 1

49:09

Got it.

S

Speaker 2

49:10

So that's essentially the. So today for example, that's the amount of engineering that take. So what they will do is they will generate an Ethereum block which will like two power 30 risk five cycles and they'll break it into shards and then they'll use some kind of like a parallel architecture to prove the Ethereum block completely end to end.

S

Speaker 1

49:29

Okay. I mean without looking at it as an Ethereum block. Right. Let's say it's like 100 independent transactions. You know what the order is? Can you do it? Like, can you create like recursive proof? Like you take the first transaction, you create a proof and then with that you.

S

Speaker 2

49:46

Yeah, so people have tried to do this in multiple ways, but the way they actually try to do this is that they take all the 100 transactions, generate all the RISC V assembly for it, take from, go from state A to state B and then essentially independently prove that the now break this into chunks of RISC 5 cycle basically. So say 100 transactions took say maybe 2 power 25 cycles of risk 5. So a transaction is like. So how would a transaction look? A transaction would look like you take all the function data of the Ethereum contract, you take all the, you take the initial account states for

whatever the transaction touches, the post account states for whatever happens after the transaction is executed.

S

Speaker 2

50:35

Now you, inside your execution environment, you essentially go from state A to state B, check that the pre state was consistent with your Merkle root Check. Your post state was consistent with the next Merkle root. And you keep doing this iteratively say for 1000 transactions. Now this entire process took say 2 power 25 RISC 5 cycles. Now you can only prove 2 power 20 cycle. So now you break it into shards. You break it into shards of 2 power 20. So you will be left with 32 shards. Now in 32 shards you say that the consecutive shards are correct. So you prove each of them shards independently. And now you have a prover that sits on top of this that says that these are actually consecutive shards.

S

Speaker 2

51:15

So that you want to basically show that the memory is consistent across the shards and the PC program counter is consistent across the shards. And now you basically build this tree out and you now do this layer and you will be fine. You finally get the proof on the top is essentially given to the smart contract on the L1 and the proof on the usually a Grot 16.

S

Speaker 1

51:37

And so this you could also parallelize with the GPU.

S

Speaker 2

51:40

Yes, yes, exactly. This is where essentially the GPUs are used or like the core. So they left cluster cores. So most of them would use like a 200 cluster cores or 400 cluster cores to actually like do this parallelly. And this one on the top is unique because that will basically be a Grot 16 Prover because that's what you can prove on an L1.

S

Speaker 1

52:02

Got it, got it. The growth system. Because like that's like can you prove a stark proof today on Ethereum L1 you or you have to.

S

Speaker 2

52:12

So yeah. So the thing is that they prove it with like reduced security and they pay about 4 million of gas or 5 million of gas on.

S

Speaker 1

52:24

But it's actual stock prover, right?

S

Speaker 2

52:26

Not it's an actual stock prover of kvm.

S

Speaker 1

52:30

Okay, okay.

S

Speaker 2

52:31

So it's a KAROVM stock.

S

Speaker 1

52:33

If you do like, if you want to prove like a risk zero.

S

Speaker 2

52:37

Yeah. So that they don't do they wrap it up inside a Grot 16 and then they will.

S

Speaker 1

52:41

Okay, okay, that's got it.

S

Speaker 2

52:44

And the problem is nobody has vetted it out. So nobody has audited. I don't know how much auditing has gone one into basically. So there is a non native aspect of this computation. So when you go from start to Grot 16, your field basically changes. So you need to now execute certain field computations that were happening over the small field over a big field. So this requires careful circuit wiring which I don't know whether like there are. You could have errors in it.

S

Speaker 1

53:12

Got it. I'm Assuming everyone uses the same wrapper.

S

Speaker 2

53:16

Yeah.

S

Speaker 1

53:16

And they want.

S

Speaker 2

53:17

And there have been bugs actually been found in things. So it's a. It's a long story. But anyway, so yeah, this is essentially the way people prove today.

S

Speaker 1

53:27

Okay. So I think. Okay fine. So on. On this like are we saying the inclusion proofs are a hard requirement?

S

Speaker 2

53:35

Yeah.

S

Speaker 1

53:35

Zika proofs. I see why they are required.

S

Speaker 2

53:38

Yeah.

S

Speaker 1

53:39

Because if you want to avoid re execution.

S

Speaker 2

53:42

Yes.

S

Speaker 1

53:44

You need CK proofs.

S

Speaker 2

53:46

Yes.

S

Speaker 1

53:47

But inclusion proofs are slight. It doesn't look like it's conclusive that we either want it or not want it. Right.

S

Speaker 2

53:55

Yes.

S

Speaker 1

53:55

So inclusion proofs are not okay with inclusion proof if it doesn't add a scaling bottleneck.

S

Speaker 2

54:01

Yeah. So I am also not very keen on inclusion proofs. But right now I have not found a solution that does DB state that just like for batch updates except for Verkal trees or Merkle Patricia trees or a Merkle trees or Smart Merkle trees. Which implies that you get inclusion proof for free.

S

Speaker 1

54:22

Yeah.

S

Speaker 2

54:23

Inclusion proof is like sort of being. We are arriving it as a byproduct right now. But tomorrow if we can sort of somehow come up with an accumulation scheme that does not.

S

Speaker 1

54:34

But these. These Mercury trees also add like cost overhead. Right. So it's a trade off. Right now it's. What is unclear is that does that exist accumulator scheme that is also ZK Approvable.

S
Speaker 2
54:48
Yes, exactly. It is.
S
Speaker 1
54:50
And also fast enough.
S
Speaker 2
54:52
Yes, exactly. Exactly. This is exactly the question that needs to be answered. Actually after looking at that last lattice paper I actually want. I. I will ask Justin whether this can be done with Poseidon as I'm not positive about it but just let's. Let's see if he say if he knows something.
S
Speaker 1
55:08
Yeah.
S
Speaker 2
55:09
If he knows something.
S
Speaker 1
55:10
That the major assumption there is that one whatever that if you look at the code they actually they extend the lake 3/7 to a/ extension and then that's what they use to fill the 1024.
S
Speaker 2
55:28
Yeah.
S

Speaker 1
55:30
Long lattice. Right.
S
Speaker 2
55:31
Yeah.
S
Speaker 1
55:32
And they also expect the U16 to behave like a lattice. I haven't checked it but the assumption that is that you U16 is a lot behaves as a lattice.
S
Speaker 2
55:43
I think that's. I don't think that's. I think that's true if I'm not as long as integers if my understanding of lattices. But I've. I've forgotten like I did it during my grad 30 here short destructive problem. I. It's actually a very nice solution to it. But anyway so. Yeah. So My understanding is it should be a lattice. I don't know whether it's an assumption, but I'll check.
S
Speaker 1
56:07
Yeah. What is the lattices? What is lattices? Algebraic structure. From the group, you can take two.
S
Speaker 2
56:17
Vectors and then you just need to do integer additions on two vectors. So it's basically the set generated by these two vectors.
S
Speaker 1
56:25

No, no, I meant from a group theory perspective, not the lattice cryptography. But you have ring fields, right? So.

S

Speaker 2

56:33

So lattices are objects that are defined basically using vectors. So vectors are algebraic objects. So vectors. Vectors come from a vector space. There is an algebraic object called something called as a vector space.

S

Speaker 1

56:45

Yeah. Basically here the 1024 vector is. It's a vector in the U16 lattice space.

S

Speaker 2

56:54

Yes. So that's right. Yes.

S

Speaker 1

56:58

Okay, got it.

S

Speaker 2

56:59

I will confirm this and let you know by tomorrow. If my. If this is. I'll just read in detail. I just skimmed through it. But mine. Like I was more interested in the algorithm so that I can. To know whether it can.

S

Speaker 1

57:12

The algorithm is actually very simple.

S

Speaker 2

57:15

You just Blake hash it and then.

S

Speaker 1

57:16

Basically you generally make. Yeah, you do you support both operations. So a mix in and a mix out.

S

Speaker 2

57:24

Yes.

S

Speaker 1

57:25

The idea is that you can remove it and it will still retain the operation because like it's both. It. I also review this that it's both commutative, one dissociative. So you get those properties as well.

S

Speaker 2

57:39

Yeah. So I will check this and let you know by tomorrow. Can you. Would it be okay if I ask you a favor? Let me design like. I'll ask. I'll sit with my engineer and I'll design like a Rust execution environment, a very minimal one. So the token contract that we discussed, like the mint thing and like the DB state for now I think let's stick to Merkle trees or Verkle trees for now at least for the gff. And then we can swap it out with whatever accumulator scheme that comes out. And if I find a better way.

S

Speaker 1

58:09

No, I think we need to articulate it well as to why this is a consideration. Because what will once you ship it set in stones like the.

S

Speaker 2

58:20

Yes, yes, I agree. So essentially what. What I was planning to do is that I will write an. I will write a proposal, I'll share it with you and then you can also go over it and then we can just share it over email with them. Does that.

S

Speaker 1

58:36

Yeah, sounds good.

S

Speaker 2

58:37

So I say that it is a mix of ideas that we discussed and this is what were thinking. That's fine with you.

S

Speaker 1

58:44

Yeah.

S

Speaker 2

58:44

Okay.

S

Speaker 1

58:45

Yeah, I'm okay with it. I also, I mean, we. It looks like were thinking about these problem statements, like similarly. Yes. S1. Right?

S

Speaker 2

58:54

Yes, exactly. So I. Let me. Let me draft this proposal out. I will also just include Karan in it. And then, like, we can both just. I'll just send out a mail saying that we both thought about it and this is what we are thinking about.

S

Speaker 1

59:09

Okay, that's fine. Okay, sounds good.

S

Speaker 2

59:11

I'll share the proposal with you before that so that you can also write it out.

S

Speaker 1

59:15

Oh, sounds good, man.

S

Speaker 2

59:16

Sounds good. And the last thing that.

S

Speaker 1

59:20

Another call now.

S

Speaker 2

59:22

Yeah, no problem.

S

Speaker 1

59:22

See you. But we can continue our text.

S

Speaker 2

59:24

Yes. Okay, fine.

S

Speaker 1

59:26

Okay, bye.